T15 # MCP Secrets Management

Technical Advanced

Managing MCP server credentials without exposing them

The core problem

MCP servers need API keys and credentials to function. Storing them in plaintext in `settings.json` or `.mcp.json` creates a real risk: accidental commit to Git, exposure in logs, or lateral movement after a compromise. The absolute rule is that secrets must never appear in versioned files.

Three approaches depending on context

Approach	Security	Complexity	Ideal for
OS Keychain	High	Medium	Solo devs, macOS/Linux
`.env` + `.gitignore`	Medium	Low	Small teams, prototyping
Secret Vaults	Very high	High	Enterprise, SOC 2, HIPAA

Approach 1: OS Keychain (recommended)

The secret is encrypted at rest by the OS and accessible only to authorized processes. The MCP configuration exposes only a retrieval command, never the value.

```
# Illustrative macOS command; real values entered here can reach shell history
# Prefer the interactive Keychain interface for actual secrets
security add-generic-password \
  -a "claude-mcp" -s "github-token" \
  -w "ghp_your_token_here"
```

```
{
  "mcpServers": {
    "github": {
      "command": "bash",
      "args": ["-c", "GITHUB_TOKEN=$(security find-generic-password -s 'github-token' -w) /path/to/approved-mcp-server"]
    }
  }
}
```

Linux: use `secret-tool` (libsecret, GNOME Keyring or KWallet) with the same wrapper script logic.

Approach 2: .env + .gitignore

Simple and sufficient for solo use or a small team with good discipline.

```
# Create the secrets file
cat > ~/.claude/.env << EOF
GITHUB_TOKEN=ghp_your_token
OPENAI_KEY=sk-your_key
EOF
chmod 600 ~/.claude/.env   # Restrictive permissions
echo ".env" >> ~/.claude/.gitignore
```

Claude Code does not automatically load this `.env` file. Export the required variables through a trusted launcher or secret manager before starting Claude, then reference `"${GITHUB_TOKEN}"` in the MCP `env` field. Review a shell file before sourcing it.

Template for teams: commit `mcp-config.template.json` with placeholders, generate the real file via `envsubst`. Never commit the generated file.

Approach 3: Secret Vaults (enterprise)

HashiCorp Vault, AWS Secrets Manager, or 1Password CLI enable automated rotation, centralized auditing, and granular access control. The principle remains identical: a wrapper script retrieves the secret at runtime and exports it as an environment variable before launching the MCP server.

```
# HashiCorp Vault
export GITHUB_TOKEN=$(vault kv get -field=token secret/claude/github)
/path/to/approved-mcp-server
```

What to never do

Hardcode a token directly in the `command` or `args` value of a versioned MCP server. Even in a private repo, secrets in plaintext in Git remain in the history even after deletion and can be exposed by a `git log` or unauthorized access.

A pre-commit hook that scans for token patterns (`ghp_`, `sk-`, `Bearer`) before each commit is a simple additional protection to put in place.

`CLAUDE_CODE_SUBPROCESS_ENV_SCRUB` **(v2.1.83+)**: environment variable that forces Claude Code to strip sensitive variables that Claude Code recognizes, including Anthropic/cloud credentials and credentials in package-registry URLs before passing them to subprocesses and MCP servers. It is not a universal secret-name matcher, and may break subprocesses that require those credentials. Verify the effective environment.